

Week 7

תרגילון זה מכיל 8 עמודים. וודאו שאינכם מפספסים אף עמוד.

Exercise Instructions:

During this exercise, document your progress using screenshots of the entire screen to demonstrate that you have completed all the required tasks. Save all screenshots in a folder named "proof" within your GitHub project, and organize them chronologically (e.g., 1.png, 2.png, etc.). Screenshots can be in any standard image format (e.g., png, jpeg).

You don't need hundreds of screenshots, but please include enough to clearly show completion of the required tasks. Your README file must also include a link to the repository.

For Submission:

Download the entire repository from GitHub and upload it to Moodle. Do not provide links to external locations (e.g., Google Drive or GitHub links). Note: Moodle has a 5MB upload limit. If necessary, remove screenshots from the submission to fit the size limit, but ensure that all screenshots remain in the GitHub repository.

Promises:

1. Execute the following code and explain the output:

```
console.log(1);

setTimeout(() => {
  console.log(2);
}, 0);

Promise.resolve().then(() => {
  console.log(3);
});

console.log(4);
```

2. Execute the following code and explain the output:

```
setTimeout(() => console.log(1), 0);
```

```
Promise.resolve().then(() => {  
  console.log(2);
```

```
setTimeout(() => console.log(3), 0);
```

```
  Promise.resolve().then(() => {  
    console.log(4);  
  });  
});
```

3. Execute the following code and explain the output:

```
async function demo() {  
  console.log(1);
```

```
  await Promise.resolve();  
  console.log(2);
```

```
  await Promise.resolve();  
  console.log(3);  
}
```

```
console.log(4);  
demo();  
console.log(5);
```

4. Execute the following code and explain the output:

```
async function level3() {  
  console.log(1);  
  await new Promise((resolve, reject) => {  
    console.log(2)  
    resolve()  
  });  
  console.log(3);  
}
```

```
async function level2() {
```

```
console.log(4);  
await level3();  
console.log(5);  
}
```

```
async function level1() {  
  console.log(6);  
  await level2();  
  console.log(7);  
}
```

```
level1();
```

5. Execute the following code and explain the output:

```
async function level3() {  
  console.log(1);  
  await new Promise((resolve, reject) => {  
    console.log(2)  
    resolve()  
  });  
  console.log(3);  
}
```

```
async function level2() {  
  console.log(4);  
  await level3();  
  console.log(5);  
}
```

```
async function level1() {  
  console.log(6);  
  level2();  
  console.log(7);  
}
```

```
level1();
```

MVC:

1. Implement the code from the lecture (8 - MVC example.pdf) until slide 20 (including).
2. You have to implement the code gradually. That is, you first execute only the code from the first slide. Then, implement the changes described in slides 2-3 and execute. Then, the changes in slides 5-6 and execute. Continue like this, understanding the gradual changes that each slide introduces towards MVC.
3. You have to document (using print screens) that you have indeed implemented the code gradually as instructed.
4. Use debugging to see the MVC structure "in action".

REST:

We will now transform the MVC project into RESTful API.

1. In:

```
const getArticle = (req,res) => {  
  const id = 1  
  res.render('article.ejs', { foo : articles.getArticle(id) })  
}
```

Replace:

```
res.render('article.ejs', { foo: articles.getArticle(id) })
```

With:

```
res.json(articles.getArticle(id))
```

2. Use curl (or an equivalent tool, but not a browser) to invoke the getArticle function and ensure that the response contains the relevant json object and **not** an HTML.
3. Add the necessary code to the server so that the server will support CRUD operations (create, read, update, delete) in a RESTful manner. That is:

- a. Get (to get all articles)
 - i. Simple - very similar to what we did above for getArticle
 - b. Post (to add a new article)
 - i. What needs to be added?
 1. Relevant code in routes (for post)
 2. Relevant code in controller (to extract new article's fields from the request's body, call model function, return 201)
 3. Relevant code in model (to add the new article object to the array)
 - c. PUT or PATCH (to update an article)
 - d. DELETE (to delete an article)
- e. אני מצרף בסוף התרגיל את הפתרון לסעיפים a,b, פלוס דוגמת הרצה. את הפתרון לסעיפים c,d תצטרכו להשלים בעצמכם על בסיס הקוד שנתתי.
4. Demonstrate using curl (or equivalent) that each of these CRUD operation works.

End of Week 7 Mini-Exercise

EXPLORER

...

FOO

controllers

articles.js

models

articles.js

node_modules

routes

articles.js

app.js

package-lock.json

package.json

JS app.js

JS articles.js controllers

JS articles.js models

JS articles.js routes

JS app.js > ...
1 const express = require('express')
2 const app = express()
3 const articleRoutes = require('./routes/articles')
4
5 app.use(express.json())
6 app.use('/api/articles', articleRoutes)
7
8 app.listen(3000)
9

EXPLORER

...

FOO

controllers

articles.js

models

articles.js

node_modules

routes

articles.js

app.js

package-lock.json

package.json

JS app.js

JS articles.js controllers

JS articles.js models

JS articles.js routes

routes > JS articles.js > ...
1 const express = require('express')
2 const router = express.Router()
3 const controller = require('../controllers/articles')
4
5 router.route('/')
6 | .get(controller.getAllArticles)
7 | .post(controller.createArticle)
8
9 router.route('/:id')
10 | .get(controller.getArticleById)
11 | // .patch(controller.updateArticle)
12 | // .delete(controller.deleteArticle)
13
14 module.exports = router
--

EXPLORER

FOO

controllers

JS articles.js

models

JS articles.js

node_modules

routes

JS articles.js

app.js

package-lock.json

package.json

JS app.js

JS articles.js controllers

JS articles.js models

JS articles.js routes

controllers > JS articles.js > getArticleById > getArticleById > article

```
1  const Article = require('../models/articles')
2
3  exports.getAllArticles = (req, res) => {
4    res.json(Article.getAllArticles())
5  }
6
7  exports.getArticleById = (req, res) => {
8    const article = Article.getArticle(parseInt(req.params.id))
9    if (!article)
10     return res.status(404).json({ error: 'Article not found' })
11    res.json(article)
12  }
13
14  exports.createArticle = (req, res) => {
15    const { title, content } = req.body
16    if (!title || !content)
17     return res.status(400).json({ error: 'Title and content required' })
18
19    const newArticle = Article.createArticle(title, content)
20    res.status(201).location(`/api/articles/${newArticle.id}`).end()
21  }
```

EXPLORER

FOO

controllers

JS articles.js

models

JS articles.js

node_modules

routes

JS articles.js

app.js

package-lock.json

package.json

JS app.js

JS articles.js controllers

JS articles.js models

JS articles.js routes

models > JS articles.js > createArticle

```
1  let idCounter = 0
2  const articles = []
3
4  const getAllArticles = () => articles
5
6  const getArticle = (id) => articles.find(a => a.id === id)
7
8  const createArticle = (title, content) => {
9    const newArticle = { id: ++idCounter, title, content }
10    articles.push(newArticle)
11    return newArticle
12  }
13
14  module.exports = {
15    getAllArticles,
16    getArticle,
17    createArticle
18  }
19
```

```
% curl -i http://localhost:3000/api/articles
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 2
ETag: W/"2-19Fw4VU07kr8CvBlt4zaMCqXZ0w"
Date: Sun, 18 May 2025 21:57:56 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

Get

```
[{"id":1,"title":"hello","content":"world"}]
```

```
% curl -i -X POST http://localhost:3000/api/articles \
-H "Content-Type: application/json" \
-d '{"title":"hello", "content": "world"}'
HTTP/1.1 201 Created
X-Powered-By: Express
Location: /api/articles/1
Date: Sun, 18 May 2025 21:58:09 GMT
Connection: keep-alive
Keep-Alive: timeout=5
Content-Length: 0
```

Post

```
% curl -i http://localhost:3000/api/articles
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 44
ETag: W/"2c-wLz1NM9KqUvpeNSe3YU1o1iWyDE"
Date: Sun, 18 May 2025 21:58:15 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

Get

```
[{"id":1,"title":"hello","content":"world"}]
```

```
% curl -i http://localhost:3000/api/articles/1
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 42
ETag: W/"2a-gpTkubZhIRwUp+9yPF0tf1tzdL4"
Date: Sun, 18 May 2025 21:58:38 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

Get

```
[{"id":1,"title":"hello","content":"world"}]
```

```
% curl -i http://localhost:3000/api/articles/2
HTTP/1.1 404 Not Found
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 29
ETag: W/"1d-Ub44aXKiiJEe3tqbqvC+PRJfqto"
Date: Sun, 18 May 2025 21:58:50 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

Get

```
{"error":"Article not found"}]
```